

Taller 2: Redes Neuronales Multicapa (MLP) y Backpropagation

Informe de Resultados

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002
Fecha: 17 de septiembre de 2026

Índice

1. Introducción	2
2. Marco teórico (resumen)	2
2.1. Arquitectura del perceptrón multicapa	2
2.2. El algoritmo de retropropagación	2
2.3. Por qué la XOR necesita una capa oculta	3
2.4. Sobreajuste y parada temprana	3
3. Metodología	3
3.1. Implementación	3
3.2. Arquitecturas y parámetros evaluados	4
3.3. Datasets y particiones	4
4. Resultados	5
4.1. Problema XOR	5
4.2. Clasificación multiclase — Iris	6
4.3. Wine y Breast Cancer Wisconsin — distintas particiones	7
4.4. Análisis de sobreajuste (overfitting)	9
5. Análisis y discusión	11
5.1. Por qué el MLP resuelve la XOR y los modelos del Taller 1 no	11
5.2. Efecto del número de neuronas ocultas y de la tasa de aprendizaje	11
5.3. Proporción de partición y generalización	11
5.4. Evidencia de sobreajuste y su mitigación	12
6. Conclusiones	12
7. Referencias	13
A. Tablas complementarias	13
B. Código de los bloques completados	13

1. Introducción

El Taller 1 cerró con una limitación que ninguno de los dos modelos de una sola neurona pudo superar. El Perceptrón y el Adaline resolvieron las compuertas AND y OR, pero frente a la XOR ambos quedaron en el 25 por ciento de acierto con la semilla usada allí, porque una neurona solo puede trazar un hiperplano y la XOR necesita dos. Este taller retoma exactamente ese punto. El objetivo es implementar un perceptrón multicapa, con al menos una capa oculta y entrenado por retropropagación del error, y comprobar sobre problemas de dificultad creciente qué gana la red al agregar esa capa.

El trabajo se desarrolló sobre el *live script* base entregado por el docente, en el cual se completaron las cuatro funciones marcadas como pendientes sin alterar sus nombres ni sus argumentos. A partir de ahí se agregaron los experimentos de los numerales 3 a 6 del enunciado. Primero la XOR de dos y tres entradas, comparada contra los resultados del Taller 1. Después el problema multiclase de Iris con varias arquitecturas y tasas de aprendizaje. Luego los conjuntos Wine y Breast Cancer Wisconsin en las cuatro proporciones de partición, con el Perceptrón y el Adaline como referencia, y al final el análisis del sobreajuste y la parada temprana que lo mitiga. El informe cierra con la discusión de las cuatro preguntas que plantea la plantilla y con las conclusiones que se derivan de los datos.

2. Marco teórico (resumen)

2.1 Arquitectura del perceptrón multicapa

Un perceptrón multicapa organiza las neuronas en capas sucesivas. La capa de entrada solo distribuye el patrón, cada capa oculta calcula una suma ponderada de las salidas de la capa anterior y le aplica una función de activación, y la capa de salida entrega la respuesta de la red. Para la neurona j de la capa k el cálculo es el de la ecuación (1), donde $a^{(0)}$ es la entrada x .

$$net_j^{(k)} = \sum_i w_{ji}^{(k)} a_i^{(k-1)} + b_j^{(k)}, \quad a_j^{(k)} = f\left(net_j^{(k)}\right) \quad (1)$$

Este recorrido desde la entrada hasta la salida se llama propagación hacia adelante. La diferencia con la neurona única del Taller 1 está en que la función f de las capas ocultas debe ser derivable y no lineal. El taller ofrece tres opciones, la sigmoideal $1/(1 + e^{-net})$ que produce valores entre cero y uno, la tangente hiperbólica que produce valores entre menos uno y uno, y la ReLU, que vale net cuando es positivo y cero en caso contrario.

2.2 El algoritmo de retropropagación

La red se entrena minimizando el error cuadrático $E = \frac{1}{2} \sum (d(x) - Y(x))^2$ por descenso de gradiente, igual que el Adaline. La novedad es que ahora hay pesos en capas que no ven directamente el error, y para corregirlos hace falta repartir ese error hacia atrás. El reparto se hace con un término δ por neurona. En la capa de salida el término se calcula con el error observado, ecuación (2), y en cada capa oculta se obtiene a partir de los δ de la capa siguiente pesados por las conexiones que las unen, ecuación (3).

$$\delta^{(L)} = (d(x) - Y(x)) \odot f'\left(net^{(L)}\right) \quad (2)$$

$$\delta^{(k)} = \left(W^{(k+1)T} \delta^{(k+1)}\right) \odot f'\left(net^{(k)}\right) \quad (3)$$

Con los δ calculados, cada peso se corrige en proporción al δ de la neurona que alimenta y a la activación que recibe, ecuación (4). El coeficiente de momento β agrega una fracción del incremento anterior, lo cual acelera el avance cuando los incrementos sucesivos apuntan en la misma dirección.

$$\Delta W^{(k)} = \eta \delta^{(k)} \left(a^{(k-1)} \right)^T + \beta \Delta W_{\text{anterior}}^{(k)}, \quad W^{(k)} \leftarrow W^{(k)} + \Delta W^{(k)} \quad (4)$$

Cuando la red tiene una sola capa y activación lineal, las ecuaciones (2) y (4) se reducen a la Regla Delta del Taller 1. La retropropagación es entonces la extensión natural de ese algoritmo a redes con capas ocultas.

2.3 Por qué la XOR necesita una capa oculta

Una neurona con función escalón divide el espacio de entrada en dos regiones separadas por un hiperplano. La XOR de dos entradas asigna la clase uno a los puntos (0, 1) y (1, 0) y la clase cero a (0, 0) y (1, 1), y esos cuatro puntos no admiten ninguna recta que deje a un lado los de una clase y al otro los de la otra. Por eso el Perceptrón y el Adaline del Taller 1 se quedaron en el 25 por ciento con la semilla de ese taller, y en este se promedian cinco semillas. Rumelhart, Hinton y Williams (1986) mostraron que una capa oculta entrenada por retropropagación resuelve exactamente este tipo de problema. Con una capa oculta la red puede trazar dos rectas y combinarlas. Dos neuronas ocultas bastan en principio, una que se activa cuando alguna entrada vale uno y otra que se activa cuando ambas valen uno, y la neurona de salida aprende a responder uno solo cuando la primera está activa y la segunda no. La capa oculta transforma el problema en otro que sí es linealmente separable.

2.4 Sobreajuste y parada temprana

Una red con suficientes pesos y suficientes épocas puede aprender los datos de entrenamiento de memoria, incluidos su ruido y sus casos particulares. El síntoma es que el error de entrenamiento sigue bajando mientras el error sobre datos no vistos deja de bajar y empieza a subir. Ese fenómeno se llama sobreajuste (Haykin, 2009). La parada temprana lo controla vigilando el error sobre un conjunto de validación durante el entrenamiento y deteniéndolo cuando ese error lleva cierto número de épocas sin mejorar, conservando los pesos del mejor punto. El criterio de parada por épocas sin mejora que pide el numeral 2 del enunciado es exactamente ese mecanismo.

3. Metodología

3.1 Implementación

Se completaron las cuatro funciones pendientes de la plantilla. `init_mlp` construye las matrices de pesos de cada capa con valores aleatorios pequeños y los sesgos en cero, `forward_mlp` aplica la ecuación (1) capa por capa y guarda las sumas ponderadas y las activaciones que necesita el paso siguiente, `backward_mlp` calcula los δ de las ecuaciones (2) y (3) desde la salida hacia la entrada y actualiza los pesos con la ecuación (4), y `activation_deriv` entrega la derivada de cada función de activación. La activación de las capas ocultas se indica con un solo nombre para todas o con una lista con una por capa, de modo que una misma red puede combinar tangente hiperbólica y sigmoide. El código de las cuatro se reproduce en el Anexo B. La plantilla usaba la estructura `net` en el ciclo de entrenamiento sin haberla creado, así que se agregó la llamada a `init_mlp` en la sección correspondiente.

Para los experimentos se agregó una función `train_mlp` que repite el ciclo de épocas de la plantilla y además calcula en cada época el error sobre un conjunto de validación, con lo cual implementa

la parada por épocas sin mejora a través del campo **patience** de la configuración. El resto de las funciones agregadas son utilidades, la predicción de clase por el umbral de la configuración o por mayor salida, la carga de cada conjunto, la partición con semilla y el escalado al rango cero a uno, estas dos últimas tomadas del Taller 1. La partición en entrenamiento y prueba de Wine y Breast Cancer se entrega además como el script `Dataset_train_test_T2`, versión análoga del script de clase del Taller 1 que lee los dos archivos de UCI y corta en las cuatro proporciones. El Perceptrón y el Adaline del Taller 1 se incluyeron como funciones locales sin cambios, y para los problemas de más de dos clases se entrena una neurona por clase y se decide por la mayor suma ponderada.

3.2 Arquitecturas y parámetros evaluados

La Tabla 1 recoge las configuraciones probadas. Todas usan activación sigmoideal en la salida, pesos iniciales de escala 0.5 y una tasa de aprendizaje de 0.3 salvo donde se indica. En las compuertas cada configuración se repite con cinco semillas y se reporta el promedio. Las arquitecturas escritas entre corchetes tienen dos capas ocultas. El error de parada por objetivo se dejó en el valor de la plantilla, 0.001, y en el conjunto de billetes se conservaron los parámetros del Taller 1 para que la comparación fuera directa.

Tabla 1: Configuraciones de MLP evaluadas. La parada es por error objetivo o por número de épocas sin mejora en validación

Experimento	Ocultas	Activación oculta	η	Momento	Épocas	Parada
XOR	2, 4, 8, [4 4]	sigmoid, tanh, relu	0.3	0, 0.5, 0.9	5000	error 0.001
Iris	3, 6, 12, [8 4]	sigmoid	0.05, 0.1, 0.3	0	1000	error 0.001
Conjuntos reales	8	sigmoid	0.3	0	500	30 sin mejora
Sobreaajuste	2, 8, 32, 64	sigmoid	0.1	0	200 a 1500	ninguna, 50 sin mejora

3.3 Datasets y particiones

La XOR de n entradas se genera como tabla de verdad completa con salida uno cuando un número impar de entradas vale uno, con 4 patrones para dos entradas y 8 para tres. Iris tiene 150 flores, cuatro medidas y tres especies de cincuenta ejemplares, y se codifica con una salida por especie. Wine, del repositorio UCI, tiene 178 vinos de tres bodegas descritos por trece medidas químicas. Breast Cancer Wisconsin Diagnostic, también de UCI, tiene 569 tumores con treinta medidas del núcleo celular y dos clases, 357 benignos y 212 malignos. El conjunto de billetes es el mismo del Taller 1, con 1372 patrones, cuatro medidas y dos clases. La Tabla 2 resume los tamaños.

Tabla 2: Conjuntos de datos empleados

Conjunto	Patrones	Entradas	Clases	Uso
XOR de 2 entradas	4	2	2	numeral 3
XOR de 3 entradas	8	3	2	numeral 3
Iris	150	4	3	numeral 4
Wine	178	13	3	numeral 5
Breast Cancer Wisconsin	569	30	2	numerales 5 y 6
Billetes	1372	4	2	numeral 5

Las particiones se generan con la misma función del Taller 1, que mezcla los índices con una semilla fija y corta en la proporción pedida, 60-40, 70-30, 80-20 y 90-10. Las entradas se escalan al rango cero a uno con los mínimos y máximos del conjunto de entrenamiento, y esas mismas cotas

se aplican al de prueba. Cuando el entrenamiento necesita un conjunto de validación, sea para la curva de error o para la parada temprana, se aparta una quinta parte del conjunto de entrenamiento con una segunda semilla, de modo que la exactitud siempre se mide sobre una prueba que la red no vio durante el entrenamiento. El Perceptrón y el Adaline, que no usan validación, se entrenan con el conjunto de entrenamiento completo. Los tres conjuntos reales provienen del repositorio UCI (Aeberhard y Forina, 1991; Wolberg et al., 1993; Lohweg, 2013) e Iris es el conjunto clásico de Fisher (1936).

4. Resultados

4.1 Problema XOR

La Tabla 3 compara los tres modelos sobre la XOR de dos y tres entradas. El Perceptrón y el Adaline no convergen en ninguna semilla y se quedan en el 30 por ciento con dos entradas y cerca del 50 por ciento con tres, es decir por debajo o al nivel de responder siempre la misma clase, porque los pesos siguen oscilando cuando se agotan las épocas. El MLP con cuatro u ocho neuronas ocultas resuelve la XOR de dos entradas en todas las semillas, aunque necesita entre 3440 y 4120 épocas para llevar el error por debajo de 0.001 con la tasa base y sin momento. Con dos neuronas ocultas, el mínimo teórico, una de cinco semillas se queda atascada. La XOR de tres entradas resulta más exigente y con la misma configuración solo la capa de ocho neuronas converge en la mayoría de las semillas. La última fila de cada bloque adelanta el efecto del momento, que con 0.9 lleva a las cinco semillas al error objetivo en ambos problemas.

Tabla 3: XOR de dos y tres entradas. Promedio de cinco semillas. El Perceptrón y el Adaline usan los parámetros del Taller 1, el MLP la tasa 0.3 sin momento

Entradas	Modelo	Ocultas	Momento	Épocas	Convergió (%)	Exactitud (%)
2	Perceptrón	ninguna	—	100.0	0	30.0
	Adaline	ninguna	—	200.0	0	30.0
	MLP	2	0	4406.0	80	100.0
	MLP	4	0	4116.6	100	100.0
	MLP	8	0	3443.0	100	100.0
	MLP	4	0.9	435.8	100	100.0
3	Perceptrón	ninguna	—	100.0	0	47.5
	Adaline	ninguna	—	200.0	0	50.0
	MLP	2	0	5000.0	0	62.5
	MLP	4	0	4187.8	40	75.0
	MLP	8	0	3122.8	80	92.5
	MLP	4	0.9	1231.4	100	100.0

El momento y la función de activación cambian ese panorama, como muestra la Tabla 4. Con cuatro neuronas ocultas y momento 0.9 la XOR de dos entradas converge en 436 épocas en lugar de 4117, casi diez veces más rápido, y la de tres entradas pasa de converger en dos de cinco semillas a converger en todas. La tangente hiperbólica también acelera, con 880 épocas en dos entradas, mientras que la ReLU resulta poco fiable en este problema, con tres de cinco semillas convergidas en dos entradas y ninguna en tres. La red de dos capas ocultas de cuatro neuronas, con tangente hiperbólica en la primera y sigmoide en la segunda, es la más rápida de todas con momento 0.9, 149 épocas en dos entradas y 260 en tres, porque cada capa adicional multiplica las formas de combinar los hiperplanos. La Figura 1 muestra la curva de error del MLP con momento 0.9 frente a la del

Adaline, que se estabiliza cerca de 0.139 desde las primeras épocas porque ningún ajuste lineal puede bajar de ahí.

Tabla 4: Efecto del momento y de la activación oculta sobre la XOR con cuatro neuronas ocultas, y una red de dos capas ocultas con activación distinta en cada una. Promedio de cinco semillas

Entradas	Activación oculta	Momento	Épocas	Convergió (%)	Exactitud (%)
2	sigmoid	0	4116.6	100	100.0
	sigmoid	0.5	2087.6	100	100.0
	sigmoid	0.9	435.8	100	100.0
	tanh	0	879.8	100	100.0
	relu	0	2416.4	60	90.0
	[4 4], tanh y sigmoid	0.9	149.0	100	100.0
3	sigmoid	0	4187.8	40	75.0
	sigmoid	0.5	4281.2	20	95.0
	sigmoid	0.9	1231.4	100	100.0
	tanh	0	1516.0	80	90.0
	relu	0	5000.0	0	87.5
	[4 4], tanh y sigmoid	0.9	259.6	100	100.0

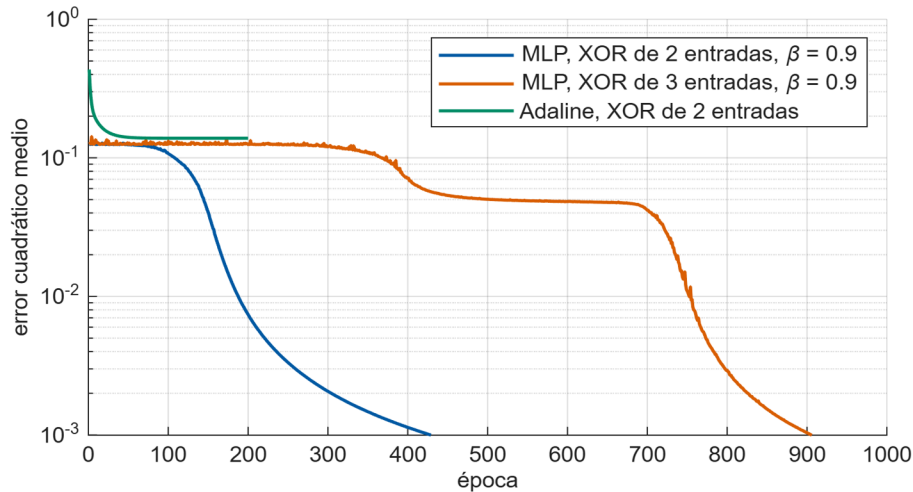


Figura 1: Curva de error de entrenamiento sobre la XOR. El MLP con cuatro neuronas ocultas y momento 0.9 llega al error objetivo, el Adaline se estanca en el mínimo alcanzable por una función lineal

4.2 Clasificación multiclase — Iris

La Tabla 5 recoge las doce combinaciones de arquitectura y tasa de aprendizaje sobre la partición 70-30, con 84 flores de entrenamiento, 21 de validación y 45 de prueba. Ninguna configuración alcanza el error objetivo de 0.001 en mil épocas, porque con tres salidas y clases que se traslapan el error cuadrático de entrenamiento se estabiliza entre 0.005 y 0.017. La exactitud de prueba es de 97.8 por ciento en nueve de las doce configuraciones, que corresponde a una sola flor mal clasificada, una *versicolor* tomada por *virginica*, y de 95.6 por ciento con la tasa 0.05 en las tres redes de una capa, que a las mil épocas todavía no terminan de ajustar la frontera entre esas dos especies y fallan una flor más. La Tabla 6 muestra la matriz de confusión de la mejor configuración, la de tres neuronas y tasa 0.1, que es la primera en alcanzar el máximo, y el Anexo A las doce.

Tabla 5: Iris sobre la partición 70-30. Mil épocas en todas las configuraciones, exactitud sobre las 45 flores de prueba

Neuronas ocultas	η	Segundos	MSE entrenamiento	MSE validación	Exactitud (%)
3	0.05	3.820	0.0159	0.0302	95.6
3	0.1	3.870	0.0114	0.0375	97.8
3	0.3	3.916	0.0106	0.0620	97.8
6	0.05	3.852	0.0166	0.0305	95.6
6	0.1	4.110	0.0115	0.0348	97.8
6	0.3	4.056	0.0066	0.0444	97.8
12	0.05	4.206	0.0167	0.0287	95.6
12	0.1	4.155	0.0117	0.0347	97.8
12	0.3	4.253	0.0095	0.0591	97.8
[8 4]	0.05	5.414	0.0109	0.0343	97.8
[8 4]	0.1	5.336	0.0068	0.0436	97.8
[8 4]	0.3	5.351	0.0053	0.0444	97.8

Tabla 6: Matriz de confusión de la mejor configuración sobre Iris, tres neuronas ocultas y tasa 0.1, 45 flores de prueba

Clase real	Predicha setosa	Predicha versicolor	Predicha virginica
setosa	14	0	0
versicolor	0	12	1
virginica	0	0	18

La Figura 2 muestra las curvas de error de entrenamiento y de validación de las doce configuraciones. La tasa gobierna la forma de la curva mucho más que el número de neuronas. Con 0.05 el error baja despacio y todavía descende al llegar a mil épocas, con 0.1 se estabiliza cerca de la época 300, y con 0.3 llega antes pero oscila de forma visible durante todo el entrenamiento, porque cada patrón mueve los pesos demasiado. Con la tasa 0.3 el error de validación además se despega del de entrenamiento y sube despacio desde la época 200, señal de un sobreajuste leve sobre 84 patrones que no llega a costar flores de prueba. La red de dos capas ocultas alcanza el error de entrenamiento más bajo en las tres tasas, pero no mejora la exactitud, porque las 45 flores de prueba ya se clasifican con una sola excepción.

4.3 Wine y Breast Cancer Wisconsin — distintas particiones

La Tabla 7 compara el MLP contra el Perceptrón y el Adaline sobre los tres conjuntos reales y las cuatro proporciones. Se agregó el conjunto de billetes porque es el que permite la comparación más directa con el Taller 1. Los valores del Perceptrón y del Adaline sobre él quedan cerca de los reportados allí y no coinciden exactamente, porque aquí el escalado usa las cotas de la parte de entrenamiento que no se reserva para validación y el orden de los patrones cambia. La Figura 3 muestra la exactitud de prueba de los tres modelos.

Los tres conjuntos cuentan historias distintas. En Wine las tres bodegas resultan separables por hiperplanos, el Perceptrón converge en todas las particiones entre 25 y 52 épocas, y aun así el MLP lo supera en tres de las cuatro, con 98.1 a 100 por ciento frente a 92.5 a 95.8, porque el error cuadrático de la capa oculta ajusta mejor la frontera que la regla de corrección por error. La excepción es la partición 90-10, donde el Perceptrón acierta los 18 vinos de prueba y el MLP falla uno. En Breast Cancer el Perceptrón nunca converge en cien épocas, señal de que las dos clases se traslapan, y el MLP lo supera en tres particiones con 97.4 a 98.2 por ciento, con la 80-20 como excepción en la

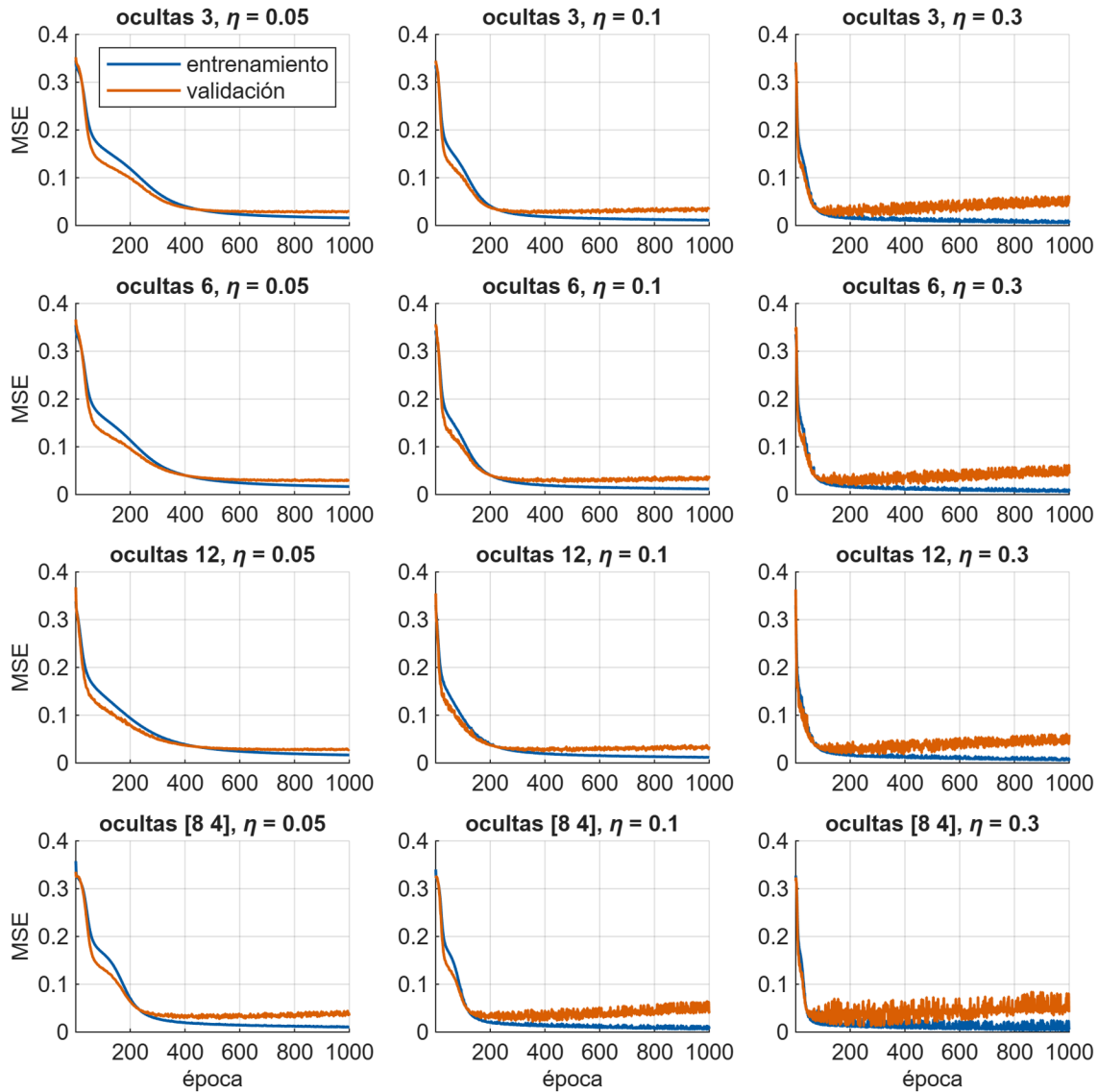


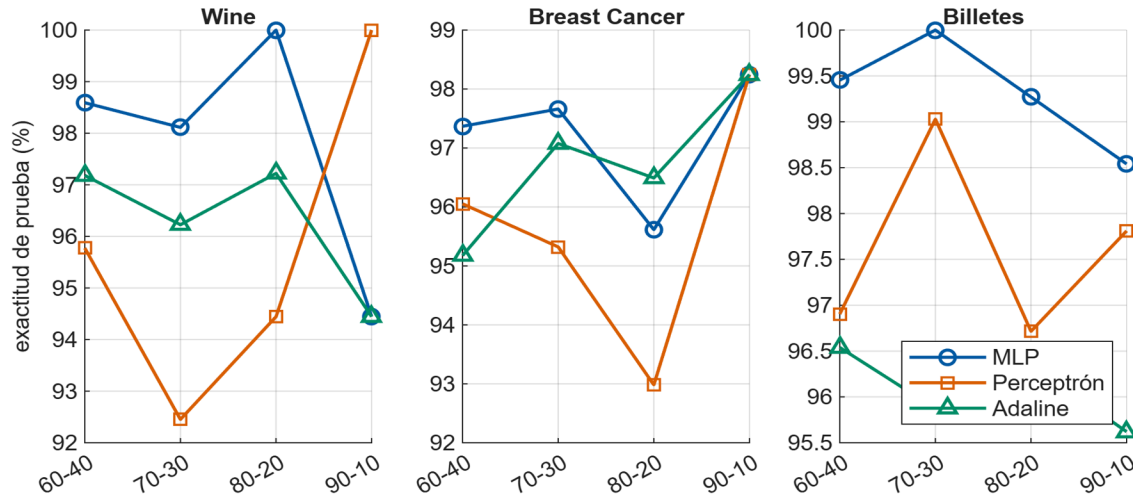
Figura 2: Curvas de error de entrenamiento y de validación sobre Iris para las cuatro arquitecturas y las tres tasas de aprendizaje

que un mal corte de la validación lo detiene temprano en 95.6. En billetes la ventaja del MLP es clara y consistente, entre 98.5 y 100 por ciento frente a un máximo de 99.0 del Perceptrón y 96.5 del Adaline. El Adaline queda en medio en la mayoría de los casos, porque el hiperplano que minimiza el error cuadrático no es el que mejor separa las clases.

El costo de esa ventaja es el tiempo. El MLP tarda entre 0.639 y 4.689 segundos por entrenamiento contra menos de 0.1 segundos del Perceptrón y el Adaline, entre veinte y ciento cuarenta veces más, porque cada patrón atraviesa dos capas hacia adelante y dos hacia atrás. En cuanto a las épocas, la parada temprana detiene al MLP entre 82 y 180 épocas, mientras que el Perceptrón y el Adaline agotan su tope en los conjuntos no separables. Sobre la proporción de partición no aparece una tendencia común, y las particiones 90-10 evalúan sobre 18, 57 y 137 patrones, así que cada error individual pesa entre 0.7 y 5.6 puntos y sus cifras son las menos estables.

Tabla 7: Exactitud de prueba según proporción de partición, con las épocas y el tiempo de entrenamiento en segundos. El MLP usa ocho neuronas ocultas y parada por treinta épocas sin mejora

Conjunto	Partición	MLP			Perceptrón			Adaline		
		%	Épocas	s	%	Épocas	s	%	Épocas	s
Wine	60-40	98.6	149	0.639	95.8	52	0.008	97.2	200	0.027
	70-30	98.1	180	0.896	92.5	32	0.007	96.2	200	0.020
	80-20	100.0	148	0.804	94.4	25	0.010	97.2	200	0.022
	90-10	94.4	151	0.877	100.0	35	0.006	94.4	200	0.036
Breast Cancer	60-40	97.4	82	0.999	96.1	100	0.014	95.2	200	0.021
	70-30	97.7	94	1.291	95.3	100	0.011	97.1	200	0.022
	80-20	95.6	102	1.761	93.0	100	0.021	96.5	200	0.029
	90-10	98.2	118	2.072	98.2	100	0.018	98.2	200	0.037
Billetes	60-40	99.5	132	3.690	96.9	100	0.022	96.5	200	0.039
	70-30	100.0	146	4.666	99.0	100	0.034	95.9	200	0.083
	80-20	99.3	125	4.689	96.7	100	0.042	96.4	200	0.068
	90-10	98.5	105	4.512	97.8	100	0.027	95.6	200	0.090

**Figura 3:** Exactitud de prueba de los tres modelos según la proporción de partición en los tres conjuntos reales

4.4 Análisis de sobreajuste (overfitting)

La Figura 4 muestra el error de entrenamiento y de validación sobre Breast Cancer durante 1500 épocas para cuatro anchos de capa oculta, sin parada temprana, y la Tabla 8 resume las cifras. Las cuatro curvas tienen la misma forma. El error de validación baja hasta un mínimo entre 0.0068 y 0.0079 en las épocas 85 a 165 y a partir de ahí sube de manera sostenida hasta terminar entre 0.0187 y 0.0220, casi el triple, mientras el error de entrenamiento sigue bajando hasta 0.0036. Ese cruce es la evidencia del sobreajuste. La red sigue ajustándose a los 318 patrones de entrenamiento y pierde capacidad sobre los 80 de validación y los 171 de prueba.

La Tabla 9 aísla el efecto del número de épocas con ocho neuronas ocultas. Cada entrenamiento más largo baja el error de entrenamiento y sube el de validación, y la exactitud de prueba cae de 95.9 por ciento a las 200 épocas a 94.7 a las 1000 y 1500, es decir dos tumores más mal clasificados por seguir entrenando.

El criterio de parada temprana implementado vigila el error de validación en cada época, guarda

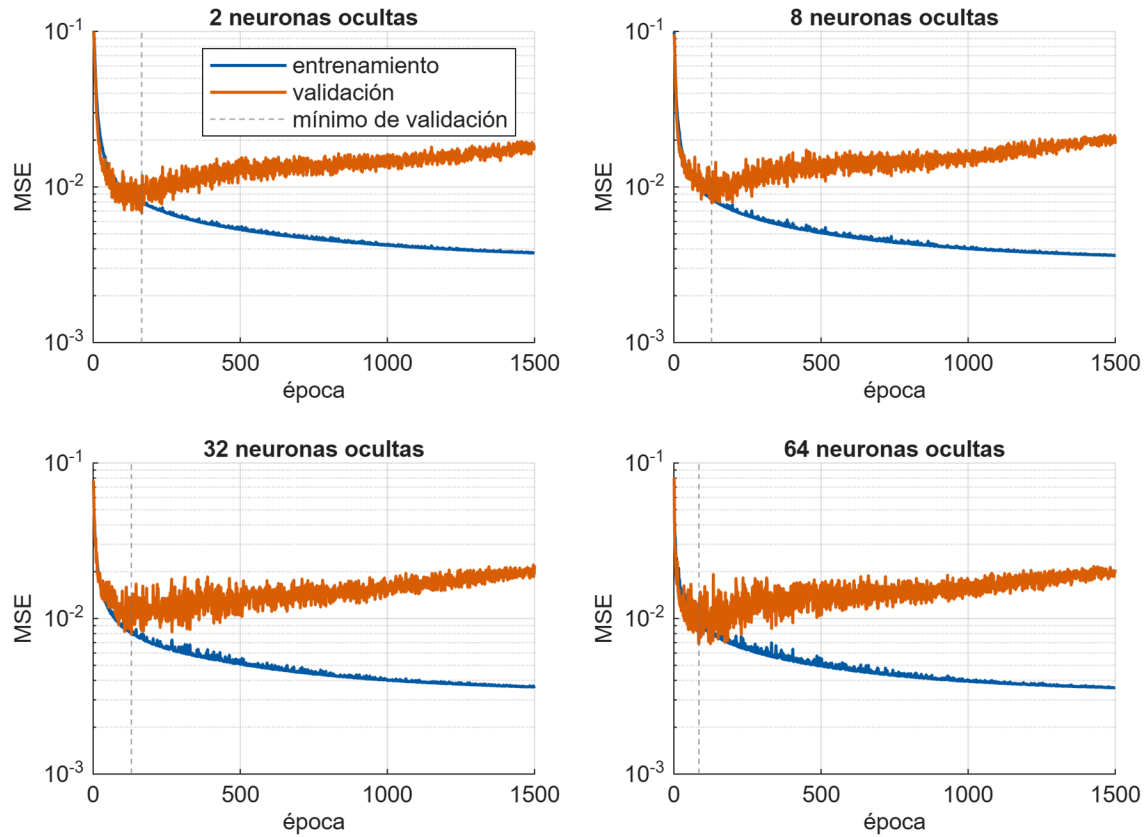


Figura 4: Error de entrenamiento y de validación sobre Breast Cancer para cuatro anchos de capa oculta. La línea discontinua marca la época de mínimo error de validación

Tabla 8: Sobreajuste en Breast Cancer, partición 70-30. Sin parada temprana el entrenamiento dura 1500 épocas, con parada temprana se detiene tras cincuenta épocas sin mejora en validación

Ocultas	Sin parada temprana, 1500 épocas					Con parada temprana	
	MSE ent.	MSE val.	Época mín.	MSE val. mín.	Exact. (%)	Épocas	Exact. (%)
2	0.0038	0.0187	165	0.0068	94.7	215	97.7
8	0.0036	0.0189	128	0.0079	94.7	178	97.7
32	0.0037	0.0220	130	0.0079	94.2	180	97.7
64	0.0036	0.0206	85	0.0068	94.7	135	97.7

Tabla 9: Efecto del número de épocas con ocho neuronas ocultas sobre Breast Cancer, sin parada temprana

Épocas	MSE entrenamiento	MSE validación	Exactitud de prueba (%)
200	0.0071	0.0115	95.9
500	0.0051	0.0126	95.3
1000	0.0040	0.0146	94.7
1500	0.0036	0.0189	94.7

los pesos del mejor punto y detiene el entrenamiento cuando pasan cincuenta épocas sin mejorar ese mínimo, devolviendo los pesos guardados. Con ese criterio el entrenamiento termina entre las épocas 135 y 215, cincuenta después del mínimo como corresponde, y la exactitud de prueba sube a 97.7 por ciento en los cuatro anchos, frente a valores entre 94.2 y 94.7 sin parada. Las cuatro redes detenidas

fallan los mismos cuatro tumores de los 171 de prueba. El número de neuronas ocultas, en cambio, apenas cambia el resultado. Con dos neuronas y con sesenta y cuatro el mínimo de validación vale lo mismo, y la red más ancha solo lo alcanza antes, de modo que en este conjunto el sobreajuste lo produce la duración del entrenamiento y no el tamaño de la red.

5. Análisis y discusión

5.1 Por qué el MLP resuelve la XOR y los modelos del Taller 1 no

Los resultados de la Tabla 3 reproducen el límite que cerró el Taller 1 y muestran cómo se supera. El Perceptrón y el Adaline quedan en 30 por ciento con dos entradas y cerca de 50 con tres, porque una sola neurona traza un solo hiperplano y ningún hiperplano separa los patrones de la XOR. El MLP con capa oculta transforma la entrada antes de decidir. Cada neurona oculta traza su propio hiperplano y la neurona de salida combina las regiones resultantes, así que el problema que llega a la salida ya es separable. La evidencia de que hacen falta las dos rectas está en la fila de dos neuronas ocultas, el mínimo teórico, donde una de cinco semillas no logra ubicarlas bien y se atasca, mientras que con cuatro u ocho neuronas sobran caminos y en dos entradas todas las semillas convergen.

La XOR de tres entradas explica por qué el problema crece rápido. Con tres entradas hay ocho vértices de un cubo alternando clase, y separarlos exige más regiones, así que con la tasa base solo la capa de ocho neuronas converge en la mayoría de las semillas. El momento resultó ser la herramienta decisiva en ese caso. Con β igual a 0.9 la corrección acumula impulso en la dirección que se repite época tras época y atraviesa las mesetas donde el gradiente es pequeño, que en la Figura 1 se ven como los tramos planos de la curva de tres entradas. El resultado es convergencia en todas las semillas y diez veces menos épocas en dos entradas.

5.2 Efecto del número de neuronas ocultas y de la tasa de aprendizaje

La tasa de aprendizaje mostró el mismo compromiso en todos los experimentos. Un valor pequeño produce curvas suaves pero lentas, como la tasa 0.05 en Iris, que a las mil épocas todavía no se ha estabilizado. Un valor grande llega antes al vecindario del mínimo pero oscila alrededor de él, como la tasa 0.3 en la Figura 2, y en Iris esa oscilación despegó el error de validación del de entrenamiento. El valor intermedio de 0.1 dio la mejor combinación de velocidad y estabilidad, y el de 0.05 dejó la frontera entre *versicolor* y *virginica* sin terminar de ajustar a las mil épocas en las redes de una capa. En la XOR la tasa base de 0.3 no fue el problema sino la ausencia de momento, que es lo que permite avanzar con paso grande sin perder estabilidad.

El número de neuronas ocultas tuvo un efecto menor de lo que se podría esperar. En Iris las cuatro arquitecturas dan la misma exactitud con las tasas 0.1 y 0.3, y en Breast Cancer las cuatro anchuras probadas caen en el mismo error de validación mínimo, solo que la más ancha llega antes. Solo en la XOR, donde la geometría exige un número mínimo de hiperplanos, la anchura marca la diferencia entre converger o no. En los conjuntos reales las clases se separan con pocas neuronas y las adicionales no aportan capacidad útil, solo más pesos que ajustar en el mismo tiempo por época.

5.3 Proporción de partición y generalización

Ninguna proporción resultó mejor de manera consistente. En Wine el Perceptrón alcanza su máximo en 90-10 y el MLP en 80-20, en Breast Cancer el MLP se mueve entre 95.6 y 98.2 por ciento sin orden con la proporción, y en billetes las cuatro particiones dan entre 98.5 y 100 por ciento. El patrón que sí se repite es la fragilidad de las particiones con conjuntos de prueba pequeños. Con 18 vinos

o 57 tumores de prueba, un solo error cambia la exactitud en varios puntos, y por eso el 100 por ciento del Perceptrón en Wine 90-10 o del MLP en billetes 90-10 dicen menos de lo que parecen. Las particiones 70-30 y 80-20 ofrecen la mejor lectura, con suficientes patrones para entrenar y suficientes para que la exactitud de prueba sea estable.

5.4 Evidencia de sobreajuste y su mitigación

La evidencia está en la Figura 4 y en la Tabla 9. En los cuatro anchos el error de validación toca un mínimo entre las épocas 85 y 165 y después sube de forma sostenida, mientras el de entrenamiento baja sin parar. A las 1500 épocas la red clasifica peor los datos nuevos que a las 200, con una diferencia de 1.2 puntos de exactitud y de 2.9 a 3.5 puntos frente a la red detenida a tiempo, aunque su error sobre los datos que ya vio sea cinco veces menor que el de validación. Ese divorcio entre los dos errores es la definición operativa del sobreajuste.

La parada temprana lo mitigó en los cuatro casos y los llevó al mismo 97.7 por ciento. El criterio es sencillo y no exige ninguna decisión previa sobre el número de épocas, porque es el propio conjunto de validación el que la toma. Lo que sí exige es apartar datos para validar, que aquí fueron 80 de los 398 patrones de entrenamiento, con lo cual la red aprende con menos ejemplos pero la exactitud sobre la prueba mide de verdad lo que la red no vio. La partición 80-20 de Breast Cancer en la Tabla 7 muestra el otro lado de ese precio, cuando el corte de validación resulta poco representativo la parada llega antes de tiempo. Un hallazgo que conviene subrayar es que aumentar las neuronas ocultas de dos a sesenta y cuatro no adelantó ni agravó el sobreajuste. La causa en este conjunto es el exceso de épocas, y el remedio correcto es detener el entrenamiento a tiempo y no reducir la red.

6. Conclusiones

El perceptrón multicapa resolvió la XOR de dos entradas con exactitud del cien por ciento en todas las semillas con cuatro u ocho neuronas ocultas, y la de tres entradas con la misma exactitud al agregar momento 0.9, mientras que el Perceptrón y el Adaline del Taller 1 quedaron en 30 y 50 por ciento sobre los mismos patrones. La capa oculta es la diferencia, y dos neuronas ocultas bastan en principio aunque en la práctica una de cinco semillas se atasca con ellas.

El momento fue el parámetro de mayor efecto sobre la velocidad. Con β igual a 0.9 la XOR de dos entradas pasó de 4117 a 436 épocas y la de tres entradas pasó de converger en dos semillas a converger en las cinco. La tangente hiperbólica también aceleró la convergencia y la ReLU resultó poco fiable en un problema tan pequeño.

Sobre Iris las doce configuraciones alcanzaron 95.6 o 97.8 por ciento de exactitud, y la tasa de aprendizaje determinó la forma de la curva de error más que el número de neuronas o de capas. La tasa 0.1 dio el mejor equilibrio entre velocidad y estabilidad.

En los conjuntos reales el MLP igualó o superó al Perceptrón en diez de las doce particiones y al Adaline en nueve, con la ventaja más clara en billetes, entre 98.5 y 100 por ciento, y en Wine, donde ganó en tres de cuatro particiones. Las dos excepciones cayeron en particiones con prueba pequeña o con un corte de validación poco representativo. El precio fue un tiempo de entrenamiento entre veinte y ciento cuarenta veces mayor.

El sobreajuste apareció en Breast Cancer como una subida sostenida del error de validación desde las épocas 85 a 165 mientras el de entrenamiento seguía bajando, costó 1.2 puntos de exactitud entre 200 y 1500 épocas, y fue independiente del ancho de la capa oculta entre dos y sesenta y cuatro neuronas. La parada temprana por cincuenta épocas sin mejora lo corrigió y elevó la exactitud de prueba de entre 94.2 y 94.7 a 97.7 por ciento en los cuatro casos.

Las proporciones 70-30 y 80-20 dieron las lecturas más estables. Las particiones 90-10 evalúan sobre muy pocos patrones y sus cifras extremas, incluidos varios cien por ciento, no deben tomarse como mejoras reales.

7. Referencias

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Haykin, S. (2009). *Neural Networks and Learning Machines* (3.^a ed.). Pearson.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Nielsen, M. (2015). *Neural Networks and Deep Learning*. Determination Press.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
- Aeberhard, S., & Forina, M. (1991). Wine. UCI Machine Learning Repository.
- Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository.
- Lohweg, V. (2013). Banknote Authentication. UCI Machine Learning Repository.

A. Tablas complementarias

Tabla 10: Matrices de confusión de las doce configuraciones sobre Iris. Cada celda es el número de flores de la clase real en fila asignadas a la clase en columna, con las clases en el orden setosa, versicolor, virginica

Ocultas	η	se-se	se-ve	se-vi	ve-se	ve-ve	ve-vi	vi-se	vi-ve	vi-vi
3	0.05	14	0	0	0	11	2	0	0	18
3	0.1	14	0	0	0	12	1	0	0	18
3	0.3	14	0	0	0	12	1	0	0	18
6	0.05	14	0	0	0	11	2	0	0	18
6	0.1	14	0	0	0	12	1	0	0	18
6	0.3	14	0	0	0	12	1	0	0	18
12	0.05	14	0	0	0	11	2	0	0	18
12	0.1	14	0	0	0	12	1	0	0	18
12	0.3	14	0	0	0	12	1	0	0	18
[8 4]	0.05	14	0	0	0	12	1	0	0	18
[8 4]	0.1	14	0	0	0	12	1	0	0	18
[8 4]	0.3	14	0	0	0	12	1	0	0	18

B. Código de los bloques completados

Se reproducen completos los cuatro bloques que la plantilla dejaba pendientes y la función de entrenamiento con parada temprana que se agregó para los experimentos. Ningún nombre ni argumento de las funciones de la plantilla fue modificado.

Listing 1: Inicialización de pesos y sesgos

```

1 function net = init_mlp(cfg)
2     % Inicializa pesos y sesgos de todas las capas en la estructura `net`.
3     net = struct();
4     net.W = {};
5     net.b = {};
6     net.dW_prev = {};
7     net.db_prev = {};
8     layer_sizes = [cfg.n_inputs, cfg.hidden_layers, cfg.n_outputs];
9     for k = 1:numel(layer_sizes) - 1
10         net.W{k} = randn(layer_sizes(k+1), layer_sizes(k)) * cfg.init_scale;
11         net.b{k} = zeros(layer_sizes(k+1), 1);
12         net.dW_prev{k} = zeros(size(net.W{k}));
13         net.db_prev{k} = zeros(size(net.b{k}));
14     end
15 end

```

Listing 2: Propagación hacia adelante

```

1 function [y, cache] = forward_mlp(net, x, cfg)
2     % Propaga la entrada x a través de todas las capas de la red.
3     a{1} = x;
4     L = numel(net.W);
5     for k = 1:L
6         netv{k} = net.W{k} * a{k} + net.b{k};
7         if k == L
8             a{k+1} = activation(netv{k}, cfg.act_output);
9         else
10            a{k+1} = activation(netv{k}, hidden_act(cfg, k));
11        end
12    end
13    y = a{end};
14    cache.a = a;
15    cache.netv = netv;
16 end
17
18 function tipo = hidden_act(cfg, k)
19     % Activación de la capa oculta k, una para todas o una por capa si act_hidden es celda.
20     if iscell(cfg.act_hidden), tipo = cfg.act_hidden{k}; else, tipo = cfg.act_hidden; end
21 end

```

Listing 3: Retropropagación del error y actualización con momento

```

1 function net = backward_mlp(net, cache, x, d, cfg)
2     % Calcula los delta de cada capa y actualiza los pesos (ver fórmulas
3     % de la Sección 1 de este mismo script).
4     L = numel(net.W);
5     delta = cell(1, L);
6     delta{L} = (d - cache.a{L+1}) .* activation_deriv(cache.netv{L}, cfg.act_output);
7     for k = L-1:-1:1
8         delta{k} = (net.W{k+1}' * delta{k+1}) .* activation_deriv(cache.netv{k}, hidden_act(cfg, k));
9     end
10    for k = 1:L
11        dW = cfg.eta * delta{k} * cache.a{k}' + cfg.momentum * net.dW_prev{k};
12        db = cfg.eta * delta{k} + cfg.momentum * net.db_prev{k};
13        net.W{k} = net.W{k} + dW;
14        net.b{k} = net.b{k} + db;
15        net.dW_prev{k} = dW;
16        net.db_prev{k} = db;
17    end
18 end

```

Listing 4: Derivadas de las funciones de activación

```

1 function dy = activation_deriv(net_in, tipo)
2     % Derivada de la función de activación, evaluada en net_in.
3     switch tipo
4         case 'sigmoid'
5             f = 1 ./ (1 + exp(-net_in));
6             dy = f .* (1 - f);
7         case 'tanh'
8             dy = 1 - tanh(net_in).^2;
9         case 'relu'
10            dy = double(net_in > 0);
11         case 'linear'
12            dy = ones(size(net_in));
13         otherwise
14             error('Función de activación no reconocida: %s', tipo);
15     end
16 end

```

Listing 5: Entrenamiento con error de validación y parada por épocas sin mejora

```

1 function [net, hist] = train_mlp(Xtr, Dtr, Xva, Dva, cfg)
2     % Mismo ciclo de la sección 8, con error de validación por época y parada
3     % temprana cuando la validación lleva cfg.patience épocas sin mejorar.
4     rng(cfg.seed);
5     net = init_mlp(cfg);
6     hist.train = zeros(cfg.max_epochs, 1); hist.val = hist.train;
7     mejor = Inf; sin_mejora = 0; mejor_net = net;
8     for epoch = 1:cfg.max_epochs
9         for p = randperm(size(Xtr, 1))
10            [~, cache] = forward_mlp(net, Xtr(p, :)', cfg);
11            net = backward_mlp(net, cache, Xtr(p, :)', Dtr(p, :)', cfg);
12        end
13        hist.train(epoch) = mse_mlp(net, Xtr, Dtr, cfg);
14        if ~isempty(Xva), hist.val(epoch) = mse_mlp(net, Xva, Dva, cfg); end
15        if hist.train(epoch) <= cfg.target_error, break; end
16        if cfg.patience > 0 && ~isempty(Xva)
17            if hist.val(epoch) < mejor
18                mejor = hist.val(epoch); sin_mejora = 0; mejor_net = net;
19            else
20                sin_mejora = sin_mejora + 1;
21                if sin_mejora >= cfg.patience, net = mejor_net; break; end
22            end
23        end
24    end
25    hist.train = hist.train(1:epoch); hist.val = hist.val(1:epoch); hist.epochs = epoch;
26 end

```